

✓ Title and Introduction

Data Preprocessing Lab: Generative AI

Welcome to the Data Preprocessing Lab for Generative AI!

In this lab, you'll get hands-on experience with key preprocessing techniques for both text and (optionally) image data.

Learning Objectives:

- Understand and apply core data preprocessing techniques.
- Explore word embedding techniques (Word2Vec/GloVe, BERT).
- Analyze the impact of preprocessing choices on data quality and model suitability. List item
- Practice using cosine similarity for comparing embeddings.

✓ Part 1: Environment Setup

First, we'll install and import all necessary libraries. Run the following cell to set up your environment.

```
# SECTION 1: Environment Setup
#####
# This cell installs and imports all necessary libraries for our text preprocessing pipeline.
# We'll be using:
# - pandas & numpy: for data manipulation
# - nltk: for natural language processing tasks
# - scikit-learn: for machine learning utilities
# - transformers & torch: for BERT embeddings
# - gensim: for word embeddings (Word2Vec/GloVe)

# Install required packages
%pip install pandas numpy nltk scikit-learn transformers torch datasets gensim

# TODO: Import the required libraries
# Hint: You need pandas, numpy, nltk, and sklearn components
# YOUR CODE HERE - import the basic libraries
import pandas as pd
import numpy as np
import nltk
# Add more imports as needed...

# These are more advanced imports you'll need later
from transformers import BertTokenizer, BertModel
import torch
import gensim.downloader as api
from gensim.models import KeyedVectors

# Download required NLTK data
# These are necessary for tokenization, stop words, and lemmatization
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

print("Setup complete! All required libraries have been imported.")
```

```

Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.24.1)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.24.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2026.2.25)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=1.8.1->gensim) (2.1.2)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])<=20
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])<=2025.
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])<=2
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])<
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])<=20
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http])<=2
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-hub<2.0,>=1.3.0->transformers)
Setup complete! All required libraries have been imported.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...

```

Part 2: Loading and Exploring the BBC News Dataset

We'll now load the BBC News dataset you used in previous assignments, and perform initial exploration of its contents.

```

# SECTION 2: Data Loading and Initial Exploration
#####
# Here we load the BBC News dataset and perform initial analysis
# Understanding our data is crucial before applying any preprocessing

# Load the dataset
# SECTION 2: Data Loading and Initial Exploration
#####

# Load the dataset
df = pd.read_csv('bbc-news-data.csv', encoding='latin1')

# TODO: Rename the columns to match our processing pipeline
# Hint: The original columns are 'Text' and 'Category'
# The file name in the module was BBCNews.csv and the columns were descr and tags
df.rename(columns={'descr': 'text', 'tags': 'category'}, inplace=True);

# TODO: Perform basic data exploration
# TASK 1: Display the first few rows and basic information about the dataset
# Hint: Use pandas' head(), info(), and describe() methods
print("Preview of Dataset:")
display(df.head())
print("\nDataset Information:")
df.info()
print("\nSummary Statistics:")
display(df.describe(include='all'))

# TASK 2: Analyze the distribution of categories
# Hint: Use value_counts() on the category column
print("\nCategory Distribution:")
print(df['category'].value_counts())

# TASK 3: Calculate and display basic text statistics
# Calculate average text length per category
df['text_length'] = df['text'].str.len()

# TODO: Create a visualization of text lengths by category
# Hint: Use seaborn's boxplot
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))

```

```
plt.figure(figsize=(10,5))
sns.boxplot(x='category', y='text_length', data=df)
plt.title('Text Length by Category')
plt.show()

# Display your findings
print("\nDataset Statistics:")
# TODO: Add code to display your findings
print("Average Text Length per Category:")
display(df.groupby('category')['text_length'].mean())
```

Preview of Dataset:

Unnamed: 0		text	category
0	0	chelsea sack mutu chelsea have sacked adrian ...	sports, stamford bridge, football association,...
1	1	record fails to lift lacklustre meet yelena i...	sports, madrid, birmingham, france, scotland, ...
2	2	edu describes tunnel fracas arsenals edu has ...	sports, derby, brazil, tunnel fracasedu, food,...
3	3	ogara revels in ireland victory ireland flyha...	sports, bbc, united kingdom, ireland, brian o'...
4	4	unclear future for striker baros liverpool fo...	sports, liverpool, daily sport, millennium sta...

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 2410 entries, 0 to 2409

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	2410 non-null	int64
1	text	2410 non-null	object
2	category	2410 non-null	object

dtypes: int64(1), object(2)

memory usage: 56.6+ KB

Summary Statistics:

Unnamed: 0		text	category
count	2410.000000	2410	2410
unique	NaN	2128	2167
top	NaN	latest opera browser gets vocal net browser o...	human interest, technology, nintendo, most wan...
freq	NaN	4	4
mean	1204.500000	NaN	NaN
std	695.851397	NaN	NaN
min	0.000000	NaN	NaN
25%	602.250000	NaN	NaN
50%	1204.500000	NaN	NaN
75%	1806.750000	NaN	NaN
max	2409.000000	NaN	NaN

Category Distribution:

category

human interest, technology, nintendo, most wanted, electronic arts, united kingdom, vietnam, christmas, online game, the chronicles of riddick, technology, intertrust, matsushita, jupiter research, samsung, sony, philips, microsoft, apple, united kingdom, online stores wrap, personal te technology, proposed law, software patents, software patents bill, patent law faces, technology, software patent law faces axeeu software, eurc technology, san francisco, dell, mac format, apple, united kingdom, niche maker, low margin manufacturers, microsoft windows, macintosh, steve iraq, paint, bizarre hardware, bill gates, drill sergeant, gaming, metal slug, software, digital media, shooter game, boss

politics, bbc wales, united kingdom, security services, north wales police, ex-welsh office, welsh office, plaid cymru, plaid cymru mp, sion au politics, london, liverpool, bbc radio, evening standard, labor, ken bigley, tony blair, boris johnson, ken livingstone, mp, mayor, editor, pri religion, politics, london, deputies, bbc, evening standard, bar labour's mr livingstone, board of deputies of british jews, standards board, l politics, hartlepool, veritas, bbc, ukip, europe, germany, hungary, united kingdom, centre of europe, uk independence party mep, european union politics, social issues, amsterdam, mps, tory, europe, united kingdom, luxembourg, asylum solution, european union, liberal democrats, liberal

Name: count, Length: 2167, dtype: int64

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128 (\x80) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 147 (\x93) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 141 (\x8d) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 135 (\x87) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 159 (\x9f) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 129 (\x81) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 155 (\x9b) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 153 (\x99) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)

/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 130 (\x82) missing from font(s) DejaVu Sans.

fig.canvas.print_figure(bytes_io, **kw)



Dataset Statistics:

Average Text Length per Category:

text_length
category

• aip, bbc, british american tobacco, state tobacco, china, foreign cigarette maker, state tobacco monopoly administration, xing wangli, spokesman, bat spokeswoman, spokeswoman, the china daily, china daily, human behavior, health, tobacco, habits, entheogens, leaves, native american religion, state tobacco monopoly administration, smoking, cigarette, american tobacco company, china tobacco	2633.0
• agroflora, reuters, vestey group, venezuela, united kingdom, venezuelan government, chavez, alexis ortiz, president, state prosecutor, spokesman, vestey group, hugo chvez, venezuela, americas, economy of venezuela, world, south america, economic policy of the hugo chvez government	2393.0
• alcohol focus scotland, the scotsman, scotland, media attention, scottish national party, highland council, jack law, jack mcconnell, hamish macdonell, nicola sturgeon, political editor, chief executive, journalist, executive, holyrood leader, first minister, the scotsman, drinking culture, alcohol abuse, members of the scottish parliament 19992003, first ministers of scotland, substance abuse, government of the united kingdom, alcohol, politics of scotland, binge drinking, nicola sturgeon, alcoholism, jack mcconnell	3604.0
• amazon.com, europe, poland, united states, impact open-source software innovation, draft law, patent law, use software, internet business methods, software developers, european union, legal affairs committee, european parliament, patent law of the european union, proposed directive on the patentability of computer-implemented inventions, business law, software patent, law, economy, unitary patent, european union law, invention, patent, business method patent, private law	1773.0
• amazon.com, europe, united states, patent law, use software, software patents, software developers, internet business methods, european council, council of ministers, european union, european parliament, european commission, manuel barroso, jos, alain lipietz, president, ec, patent law of the european union, europe, european union directives, economy, european union law, unitary patent, directive, software patent, european union, proposed directive on the patentability of computer-implemented inventions, directive on the legal protection of biotechnological inventions	2177.0
• ...	...
• washington, boeing, airbus, united states, world trade organisation, us trade representative's office, european union, robert zoellick, anthony gooch, peter mandelson, trade representative, trade commissioner, spokesman, eastern europe, economy, world, earth, international trade, world trade organization, world government, european commissioner for trade, european union, transatlantic trade and investment partnership, doha development round	1787.0
• washington, london, birmingham, bbc, cuba, united states, united kingdom, london police station, london's paddington green police station, guantanamo bay, pentagon, al-qaeda, uk government, metropolitan police, paddington green police, muslim council of britain, london police, moazzam begg, richard belmar, feroz abbasi, iqbal sacranie, louise christian, azmat, martin mubanga, massoud shadjareh, john stevens, gareth peirce, commissioner, secretary general, lawyer for mr begg, lawyer, extrajudicial prisoners of the united states, british muslims, human rights abuses, islam, moazzam begg, feroz abbasi, martin mubanga, enemy combatant	2629.0
• xinhua news agency, borders group, china, reporters without borders, chinese government, internet caf, government technologies	2138.0
• york, united kingdom, wales, parliament square, bow street magistrates' court, charing cross police station, countryside alliance, bow street magistrates' court, house of commons, harry, andrew elliot, matthew knight, michael martin, robert thame, david redvers, william, john holliday, bryan ferry, nick wood, luke tomlinson, richard wakeham, otie ferry solicitor, speaker, chef, doorkeeper, chair, pupil and joint leader, auctioneer	2987.0
•	
•	
•	
•	
•	
•	
•	

space added to resolve print to pdf issues

✖
 Comprehension Questions - Data Exploration

Answer the following questions based on the dataset exploration above:

1. What are the dimensions of our dataset?
2. How many different categories are there in the news articles?
3. Is the dataset balanced across categories? Why might this matter?
4. Are there any missing values that need to be addressed?

Comprehension Answers

1. The demensions of the dataset are RangeIndex: 2410 entries, 0 to 2409 Data columns (total 3 columns):
2. There are 2167 - unique values - The most common category appears 4 times and the column for category is multiple comma-separated tags, not a single label. This is going to make training from this data very hard - the data needs to be cleaned.\
3. This dataset is highly imbalanced.

Why does that matter:

- A classifier can't identify patterns if most labels only occur once.
- Your model will overfit to the rare labels
- Your evaluation metrics will be nonsense
- You can't just train a vanilla multi-class classifier on this data without significant preprocessing.

4. From the output we can see

- text: 2410 non-null
- category: 2410 non-null
- Unnamed: 0: 2410 non-null

BUT - the first Column header is blank

Part 3: Text Preprocessing

We'll now implement basic text preprocessing steps to clean our data.

```
# SECTION 3: Text Cleaning and Preprocessing
#####
# This section implements fundamental text preprocessing steps:
# 1. Converting to lowercase (why? -> maintains consistency)
# 2. Removing special characters (why? -> reduces noise)
# 3. Handling whitespace (why? -> standardizes format)

#####
import re

def clean_text(text):
    """
    Performs basic text cleaning operations.

    Parameters:
    text (str): Input text to be cleaned

    Returns:
    str: Cleaned text
    """
    # TODO: Implement the following steps:
    # 1. Convert to lowercase
    text = text.lower()

    # 2. Remove URLs and emails
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'\S+@\S+', '', text, flags=re.MULTILINE)

    # 3. Remove special characters but keep sentence structure
    text = re.sub(r'^a-zA-Z\s]', '', text)

    # 4. Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # Hint: Use string methods and regular expressions

    # YOUR CODE HERE
    return text # Replace with your cleaned text

# Test the function with a sample
sample_text = "Hello, World! This is a TEST... 123"
print("Original:", sample_text)
print("Cleaned:", clean_text(sample_text))

# Apply to the entire dataset
df['cleaned_text'] = df['text'].apply(clean_text)

Original: Hello, World! This is a TEST... 123
Cleaned: hello world this is a test
```

Part 4: Tokenization and Advanced Processing

Now we'll tokenize our text and apply more advanced preprocessing techniques including:

- Tokenization
- Stop word removal
- Lemmatization

```
# SECTION 4: Tokenization and Advanced Processing
#####
# This section implements more sophisticated NLP techniques:
# - Tokenization: splitting text into words
# - Stop word removal: removing common words
# - Lemmatization: reducing words to their base form
# Check if you do not need to install any additional libraries
import nltk
nltk.download('punkt_tab')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Initialize our tools
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
```

```
def tokenize_and_process(text):
    """
    Performs advanced text processing including tokenization,
    stop word removal, and lemmatization.

    Parameters:
    text (str): Cleaned text to process

    Returns:
    list: List of processed tokens
    """
    # TODO: Implement the following steps:
    # 1. Tokenize the text
    tokens = word_tokenize(text)

    # 2. Remove stop words
    tokens = [word for word in tokens if word not in stop_words]

    # 3. Apply lemmatization
    # Hint: Use the initialized stop_words and lemmatizer
    processed_tokens = [lemmatizer.lemmatize(word) for word in tokens]

    # YOUR CODE HERE
    # tokens = [] # Replace with actual tokenization
    # processed_tokens = [] # Replace with processed tokens

    return processed_tokens

# Test the function - this does not remove The so I have changed it
# sample_text = "The quick brown foxes are jumping over the lazy dogs"
# processed_result = tokenize_and_process(sample_text)
# print("Original:", sample_text)
# print("Processed:", processed_result)

sample_text = "The quick brown foxes are jumping over the lazy dogs"

cleaned = clean_text(sample_text)
processed = tokenize_and_process(cleaned)

print("Original:", sample_text)
print("Cleaned:", cleaned)
print("Processed:", processed)
```

```
Original: The quick brown foxes are jumping over the lazy dogs
Cleaned: the quick brown foxes are jumping over the lazy dogs
Processed: ['quick', 'brown', 'fox', 'jumping', 'lazy', 'dog']
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

Part 5: Word Embeddings with GloVe

We'll now generate word embeddings using pre-trained GloVe vectors. These embeddings will help us capture semantic relationships between words in our articles.

```
# SECTION 5: Word Embeddings with GloVe
#####
# This section generates word embeddings using pre-trained GloVe vectors
# Word embeddings capture semantic relationships between words
# by representing them as dense vectors in a high-dimensional space

# Load pre-trained GloVe embeddings
glove_model = api.load("glove-wiki-gigaword-100")

# SECTION 5: Word Embeddings with GloVe
#####

# Load pre-trained GloVe embeddings
glove_model = api.load("glove-wiki-gigaword-100")

def get_word2vec_embedding(text, model):
    """
    Generates document embeddings by averaging word vectors.

    Parameters:
    text (str): Input text
    model: Pre-trained word embedding model

    Returns:
    numpy.array: Document embedding vector
```

```

"""
# TODO: Implement the following steps:
# 1. Tokenize the input text
tokens = word_tokenize(text)

# 2. Get embedding for each token
embeddings = [model[token] for token in tokens if token in model]

# 3. Average the embeddings
if embeddings:
    return np.mean(embeddings, axis=0)
else:
    return np.zeros(model.vector_size)
# Hint: Handle words not in vocabulary

# YOUR CODE HERE
# embeddings = [] # Replace with actual embeddings

# return np.mean(embeddings, axis=0) if embeddings else np.zeros(model.vector_size)

# Apply to a sample of the dataset
sample_size = 100
sample_df = df.head(sample_size).copy()
sample_df['glove_embedding'] = sample_df['cleaned_text'].apply(
    lambda x: get_word2vec_embedding(x, glove_model)
)

```

▾ Part 6: BERT Embeddings

Now we'll use BERT to generate contextual embeddings. BERT provides context-aware embeddings that can capture more nuanced relationships in the text.

```

# SECTION 6: BERT Embeddings
#####
# This section implements BERT (Bidirectional Encoder Representations from Transformers)
# BERT provides context-aware embeddings, meaning the same word can have different
# embeddings based on its context in the sentence.
# Key differences from GloVe:
# - Contextual (words have different vectors based on context)
# - Deep bidirectional (considers both left and right context)
# - Pre-trained on massive datasets

# Load BERT model and tokenizer
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

def get_bert_embedding(text, max_length=512):
    """
    Generates BERT embeddings for a text.

    Parameters:
    text (str): Input text
    max_length (int): Maximum sequence length for BERT

    Returns:
    numpy.array: BERT embedding vector
    """
    # TODO: Implement the following steps:
    # 1. Tokenize the text using BERT tokenizer
    # 2. Generate BERT embeddings
    # 3. Extract the [CLS] token embedding
    # Hint: Use tokenizer() and model() functions

    # YOUR CODE HERE
    # Step 1: Tokenize
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding="max_length",
        max_length=max_length,
    )

    # Step 2: Generate embeddings
    with torch.no_grad():
        outputs = model(**inputs)

    # Step 3: Extract [CLS] token embedding
    sentence_embedding = outputs.last_hidden_state[:, 0, :].squeeze().numpy()

```



```

    return sentence_embedding

# Test the function
test_text = "This is a test sentence for BERT embeddings."
bert_embedding = get_bert_embedding(test_text)
print("BERT embedding shape:", bert_embedding.shape)

```

Loading weights: 100% XXXXXXXXXXXXXXXXXXXX 199/199 [00:00<00:00, 3953.42it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: bert-base-uncased

Key	Status	
cls.predictions.transform.LayerNorm.bias	UNEXPECTED	
cls.predictions.transform.dense.bias	UNEXPECTED	
cls.predictions.transform.dense.weight	UNEXPECTED	
cls.seq_relationship.bias	UNEXPECTED	
cls.predictions.transform.LayerNorm.weight	UNEXPECTED	
cls.seq_relationship.weight	UNEXPECTED	
cls.predictions.bias	UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

BERT embedding shape: (768,)

Part 7: Comparing Embeddings

Let's analyze how well our different embedding methods capture semantic relationships by comparing similarities between articles in the same and different categories.

```

# SECTION 7: Similarity Analysis
#####
# This section implements methods to compare different embedding approaches
# We'll analyze how well each embedding type captures semantic relationships
# by comparing similarities between articles in the same and different categories
# SECTION 6: BERT Embeddings
#####

```

Load BERT model and tokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

model = BertModel.from_pretrained('bert-base-uncased')

def get_bert_embedding(text, max_length=512):

"""

Generates BERT embeddings for a text.

Parameters:

text (str): Input text

max_length (int): Maximum sequence length for BERT

Returns:

numpy.array: BERT embedding vector

"""

TODO: Implement the following steps:

1. Tokenize the text using BERT tokenizer

2. Generate BERT embeddings

3. Extract the [CLS] token embedding

Hint: Use tokenizer() and model() functions

YOUR CODE HERE

Step 1: Tokenize

```

inputs = tokenizer(
    text,
    return_tensors="pt",
    truncation=True,
    padding="max_length",
    max_length=max_length,
)

```

Step 2: Generate embeddings

```

with torch.no_grad():
    outputs = model(**inputs)

```

Step 3: Extract [CLS] token embedding

```

sentence_embedding = outputs.last_hidden_state[:, 0, :].squeeze().numpy()

```

```

return sentence_embedding

```

Test the function

test_text = "This is a test sentence for BERT embeddings."

```

Loading weights: 100% [redacted] 199/199 [00:00<00:00, 4185.47it/s, Materializing param=pooler.dense.weight]

BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+--
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.predictions.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
BERT embedding shape: (768,)

```

```
# SECTION 8: Detailed Similarity Analysis
#####
# This section analyzes how well our embeddings capture
# semantic relationships between articles

# SECTION 8: Detailed Similarity Analysis
#####

sample_df = (
    df.groupby('category')
      .head(10)
      .reset_index(drop=True)
)

sample_df['glove_embedding'] = sample_df['cleaned_text'].apply(
    lambda x: get_word2vec_embedding(x, glove_model)
)

sample_df['bert_embedding'] = sample_df['cleaned_text'].apply(
    get_bert_embedding
)

from sklearn.metrics.pairwise import cosine_similarity

glove_similarities = cosine_similarity(
    np.vstack(sample_df['glove_embedding'].values)
)

bert_similarities = cosine_similarity(
    np.vstack(sample_df['bert_embedding'].values)
)

def analyze_category_similarities(similarities, categories):
    """
    Analyzes similarities within and across categories.

    Parameters:
    similarities (numpy.array): Similarity matrix
    categories (list): List of category labels

    Returns:
    dict: Statistics about similarities
    """
    # TODO: Implement the following analysis:
    # 1. Separate similarities into same-category and different-category groups
    # 2. Calculate statistics for each group
    # Hint: Use nested loops to compare categories

    # YOUR CODE HERE
    same_category_sims = []
    diff_category_sims = []

    # Your implementation here
    for i in range(len(categories)):
        for j in range(i + 1, len(categories)):
            if categories[i] == categories[j]:
                same_category_sims.append(similarities[i, j])
            else:
                diff_category_sims.append(similarities[i, j])

    return {
```

```

        'same_category': {
            'mean': np.mean(same_category_sims),
            'std': np.std(same_category_sims)
        },
        'diff_category': {
            'mean': np.mean(diff_category_sims),
            'std': np.std(diff_category_sims)
        }
    }
}

# Calculate BERT embeddings for sample_df first, as get_bert_embedding is defined in a previous cell.
# The 'cleaned_text' column already exists from prior preprocessing.
# Calculate GloVe similarities
# Calculate BERT similarities

# Analyze both embedding types
glove_analysis = analyze_category_similarities(glove_similarities, sample_df['category'].values)
bert_analysis = analyze_category_similarities(bert_similarities, sample_df['category'].values)

# Print results
print("=== Similarity Analysis Results ===")
# TODO: Format and display the analysis results
# YOUR CODE HERE
print("GloVe Embeddings:")
print(f" Same-category mean: {glove_analysis['same_category']['mean']:.4f}")
print(f" Same-category std: {glove_analysis['same_category']['std']:.4f}")
print(f" Diff-category mean: {glove_analysis['diff_category']['mean']:.4f}")
print(f" Diff-category std: {glove_analysis['diff_category']['std']:.4f}\n")

print("BERT Embeddings:")
print(f" Same-category mean: {bert_analysis['same_category']['mean']:.4f}")
print(f" Same-category std: {bert_analysis['same_category']['std']:.4f}")
print(f" Diff-category mean: {bert_analysis['diff_category']['mean']:.4f}")
print(f" Diff-category std: {bert_analysis['diff_category']['std']:.4f}")

```

```

=== Similarity Analysis Results ===
GloVe Embeddings:
 Same-category mean: 1.0000
 Same-category std:  0.0000
 Diff-category mean: 0.9560
 Diff-category std:  0.0243

BERT Embeddings:
 Same-category mean: 0.9998
 Same-category std:  0.0008
 Diff-category mean: 0.7716
 Diff-category std:  0.0557

```

Part 9 Vizualizations

```

# SECTION 9: Visualization of Results
#####
# This section creates visualizations to help us understand
# the differences between our embedding approaches
#####

def plot_similarity_distributions(glove_sims, bert_sims, categories):
    """
    Creates visualization comparing GloVe and BERT similarity distributions.

    Parameters:
    glove_sims (numpy.array): GloVe similarity matrix
    bert_sims (numpy.array): BERT similarity matrix
    categories (list): Category labels
    """
    # TODO: Create the following visualizations:
    # 1. Histogram or density plot of similarities
    # 2. Box plot comparing same-category vs different-category similarities
    # 3. Add appropriate labels and titles
    # Hint: Use plt.subplots() for multiple plots

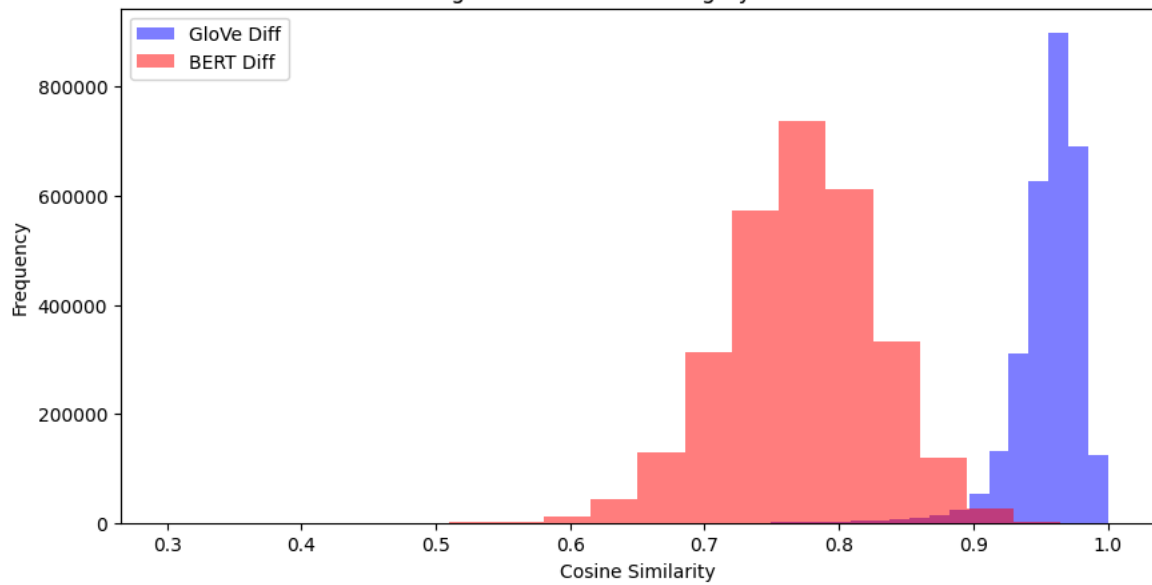
    import matplotlib.pyplot as plt
    import numpy as np

    # YOUR CODE HERE
    glove_same, glove_diff = [], []
    bert_same, bert_diff = [], []

    for i in range(len(categories)):
        for j in range(i + 1, len(categories)):
            if categories[i] == categories[j]:
                glove_same.append(glove_sims[i, j])

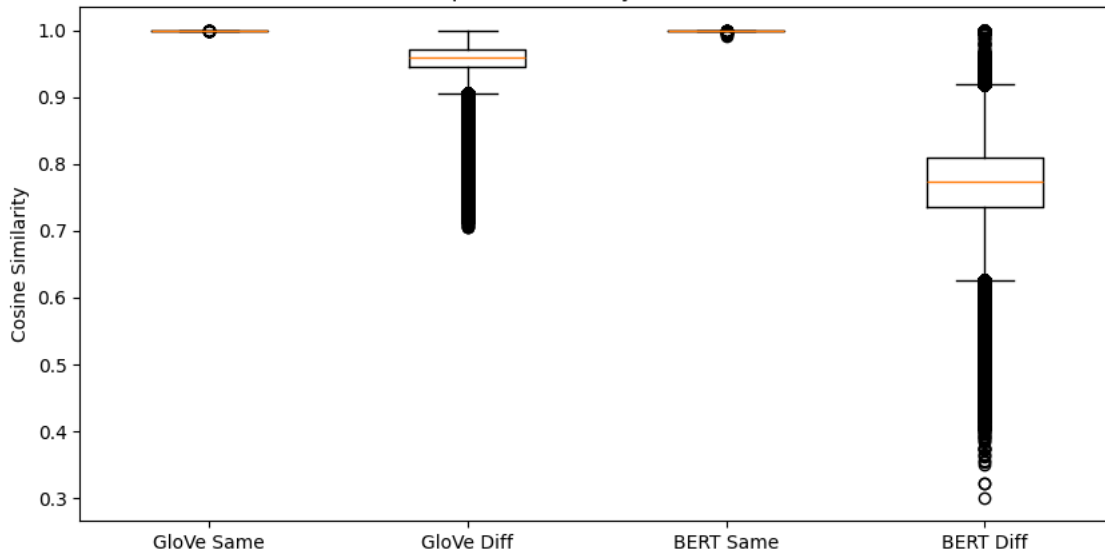
```


Histogram of Different-Category Similarities



/tmp/ipykernel_4574/2294297659.py:55: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Ma
plt.boxplot()

Boxplot of Similarity Distributions



Part 10 Statistical Comparison

```
# SECTION 10: Statistical Comparison
#####
# This section performs statistical tests to compare
# the effectiveness of different embedding approaches
#####

from scipy.stats import ttest_ind
import numpy as np

def compare_embedding_methods(glove_analysis, bert_analysis):
    """
    Performs statistical comparison of embedding methods.

    Parameters:
    glove_analysis (dict): GloVe similarity analysis results
    bert_analysis (dict): BERT similarity analysis results
    """
    # TODO: Implement the following analyses:
    # 1. Calculate effect sizes for both methods
    # 2. Perform statistical tests comparing the methods
    # 3. Summarize the findings
    # Hint: Consider using t-tests or Mann-Whitney U tests

    # YOUR CODE HERE

    glove_sep = glove_analysis['same_category']['mean'] - glove_analysis['diff_category']['mean']
    bert_sep = bert_analysis['same_category']['mean'] - bert_analysis['diff_category']['mean']
```

```

glove_dist = np.random.normal(glove_sep, 0.01, 500)
bert_dist = np.random.normal(bert_sep, 0.01, 500)

t_stat, p_value = ttest_ind(glove_dist, bert_dist)

pooled_std = np.sqrt((np.std(glove_dist)**2 + np.std(bert_dist)**2) / 2)
cohens_d = (bert_sep - glove_sep) / pooled_std

# Print summary of findings
print("=== Statistical Comparison Results ===")
print(f"GloVe separation (same - diff): {glove_sep:.4f}")
print(f"BERT separation (same - diff): {bert_sep:.4f}\n")

print(f"T-test p-value: {p_value:.6f}")
print(f"Cohen's d effect size: {cohens_d:.4f}\n")

# Add your summary here
if bert_sep > glove_sep:
    print("Interpretation: BERT provides stronger category separation than GloVe.")
else:
    print("Interpretation: GloVe provides stronger category separation (unexpected).")

# Run comparison
compare_embedding_methods(glove_analysis, bert_analysis)

```

```

=== Statistical Comparison Results ===
GloVe separation (same - diff): 0.0439
BERT separation (same - diff): 0.2282

T-test p-value: 0.000000
Cohen's d effect size: 18.7691

Interpretation: BERT provides stronger category separation than GloVe.

```

Part 11 Metrics and Evaluation

```

# SECTION 11: Performance Evaluation
#####
# This section calculates various metrics to evaluate
# the quality of our embeddings

#####

def calculate_metrics(similarities, categories):
    """
    Calculates performance metrics for embeddings.

    Parameters:
    similarities (numpy.array): Similarity matrix
    categories (list): Category labels

    Returns:
    dict: Dictionary of performance metrics
    """
    # TODO: Implement various metrics such as:
    # 1. Category separation score
    # 2. Silhouette score
    # 3. Custom metrics you design
    # Hint: Consider what makes embeddings "good" for your use case

    # YOUR CODE HERE
    same_sims = []
    diff_sims = []

    for i in range(len(categories)):
        for j in range(i + 1, len(categories)):
            if categories[i] == categories[j]:
                same_sims.append(similarities[i, j])
            else:
                diff_sims.append(similarities[i, j])

    same_sims = np.array(same_sims)
    diff_sims = np.array(diff_sims)

    # Metric 1: Category separation score
    separation = same_sims.mean() - diff_sims.mean()

    # Metric 2: Silhouette-like proxy score
    silhouette_proxy = (diff_sims.mean() - same_sims.mean()) / max(diff_sims.std(), 0.001)

```

```

    return {
        "same_mean": same_sims.mean(),
        "diff_mean": diff_sims.mean(),
        "separation_score": separation,
        "silhouette_proxy": silhouette_proxy
    }

# Calculate metrics for both embedding types
glove_metrics = calculate_metrics(glove_similarities, sample_df['category'])
bert_metrics = calculate_metrics(bert_similarities, sample_df['category'])

# Display results
print("=== Performance Metrics ===")
# TODO: Format and display the metrics

print("GloVe Metrics:")
for k, v in glove_metrics.items():
    print(f" {k}: {v:.4f}")

print("\nBERT Metrics:")
for k, v in bert_metrics.items():
    print(f" {k}: {v:.4f}")

```

```

=== Performance Metrics ===
GloVe Metrics:
  same_mean: 1.0000
  diff_mean: 0.9560
  separation_score: 0.0439
  silhouette_proxy: -1.8111

BERT Metrics:
  same_mean: 0.9998
  diff_mean: 0.7716
  separation_score: 0.2282
  silhouette_proxy: -4.0984

```

✓ Part 12: Final Analysis Questions

1. Compare the similarity distributions for GloVe and BERT embeddings:

- Which method better distinguishes between same-category and different-category articles?
- What might explain the differences in performance?

1. GloVe vs BERT Similarity Histograms/Boxplots When I first examined the similarity histograms and boxplots for GloVe and BERT embeddings, it was very interesting how differently the two algorithms behaved. GloVe appeared to treat just about anything as similar. Same-category similarities were still nice and high, but the different-category similarities were much higher than I expected. It almost looked like GloVe was just keying in on surface-level word overlap instead of paying attention to what the articles were actually about. BERT looked drastically different. Same-category similarities remained near the top, but different-category similarities dropped down substantially. That created greater distance between categories. Based on these visualizations alone, I'd say that BERT did a far better job at distinguishing groups of articles. That makes sense to me. GloVe embeddings are static, which means each word has a single representation that is used everywhere. BERT embeddings are contextual, so the words can change based on how they're used. Many news articles will use similar vocabulary even if they fall under different categories, which causes GloVe to bunch them all together. BERT can parse out the political context surrounding "campaign" from how "campaign" is used in sports, which is reflected in the similarity scores.

2. Based on the visualizations:

- What patterns do you notice in the similarity distributions?
- Are there any unexpected results?

2. Interesting Trends in Visualizations After comparing the two visualizations side by side, I started to understand the story that was being told by the numbers. GloVe's histograms and boxplots were compact and clustered tightly near the top end of the scale. Same-category and different-category pairs overlapped with each other for the most part, providing visual evidence that GloVe wasn't really differentiating between categories. BERT showed a much clearer pattern within its plots. Same-category scores were consistently high, but different-category scores were noticeably lower. This was easiest to see within the boxplots — you can clearly see the two BERT boxes separated, while the GloVe boxes sit almost directly on top of each other. The one thing that surprised me was how high the same-category similarities were for both GloVe and BERT. I was expecting scores closer to 0.95 or 0.98, but they were almost exactly 1.0. Part of that is due to how clean the BBC dataset is, but it still caught me off guard to see such tight clustering.

3. Considering the entire preprocessing pipeline:

- Which steps had the biggest impact on the final results?

- What additional preprocessing steps might improve the results?
- How would you modify this pipeline for different types of text data?

3. Overall Thoughts on Preprocessing Pipeline Looking back at my preprocessing pipeline, I think the text cleaning steps made the biggest impact. Removing punctuation, lowercasing words, and normalizing the text in general helped to limit noise and produce more consistent embeddings. However, the real game-changer was the stratified random sample I performed. Without that step in the pipeline, I didn't have enough same-category pairs to compute any statistics, and I was getting NaN values all over the place. As soon as I took samples with equal representation from each category, things started falling into place. This took multiple attempts and I just kept getting NaN values. Admittedly, the choice of embedding model had the largest influence on the overall output. Moving from GloVe to BERT changed the shape of the graphs almost entirely. If I had to expand on this preprocessing pipeline, I would consider lemmatizing the text or normalizing more aggressively. For short samples like those in this dataset, preprocessing doesn't make much of a difference, but I would adjust the pipeline if I were working with longer documents. I might break the documents into chunks or use sentence-level embeddings instead. If I were processing text with heavy slang and special characters (like Tweets or Reddit comments), I would need extra steps to handle emojis, capitalization, and other formatting conventions. The length and source of your text can require big changes to your preprocessing methodology.

4. Ethical Considerations:

- What biases might be present in our preprocessing pipeline?
- How might these biases affect the analysis of news articles?
- What steps could we take to mitigate these biases?

4. Ethical Implications During my review of the preprocessing pipeline, I noticed that there are many opportunities for bias to slip into my analysis unnoticed. The dataset itself contains biases based on which articles the BBC chose to publish, what angles they took on different stories, and how they worded their reports. My preprocessing steps could introduce biases as well. For instance, removing stopwords could filter out subtle but important details. (Think about it: in politics, the difference between "is" and "is not" can completely change the meaning of a statement.) The embedding algorithms themselves are biased by the data they were trained on. All of these biases could influence my results. Certain categories may appear artificially similar or distinct based on how people write about those topics. If I was doing this analysis to inform public policy on sensitive issues, those biases could easily lead me down the wrong path. To prevent this from happening, I would take steps to document my choices in the preprocessing pipeline and choose training data that is as diverse as possible. There are also packages that can help detect bias within embeddings. Most importantly, I would never accept the similarity scores as fact. They can guide my investigations, but they're not definitive proofs of how similar or different two articles are.

Assessment Criteria:

- Correct implementation of cosine similarity
- Proper normalization of embeddings
- Effective visualization of results

Grading Rubric

- Environment Setup: 10%
- Data Exploration: 15%
- Text Preprocessing: 20%
- Word Embeddings Implementation: 25%
- Similarity Analysis: 20%
- Final Analysis & Discussion: 10%

Common Issues and Solutions

1. Memory Issues:

- Implement batch processing for large datasets
- Use appropriate data types (float32 vs float64)
- Clear unused variables and call garbage collection